CT2 System Workflow And Process Guide

Core Transaction 2: Back-Office and Agency Management System

Audience: owners, operators, deployment engineers, QA leads, and support staff

Application root: `ct2_back_office/`

Entry point: `ct2_back_office/ct2_index.php`

Release workflow: `.github/workflows/ct2_cpanel_release_pipeline.yml`

Primary hosting model: validated artifact deployment to cPanel with shared config and storage

1. Executive Overview

CT2 is the back-office operating layer for travel-and-tours processing. It gives staff a controlled browser and API surface for operational work across agents, suppliers, approvals, availability, marketing, visa operations, staff administration, and financial reporting. CT2 is not the owner of every business record in the wider ERP. It is designed to reference upstream records where necessary while keeping ownership of its own workflow, approval, audit, reporting, and operational state.

In practice, CT2 acts as the controlled execution layer between source systems and human operators. The system is optimized for internal users, not for anonymous public traffic, and its deployment and release process is built around a strict validation gate, artifact packaging, and cPanel deployment over SSH.

2. System Boundaries

What CT2 owns

- Back-office workflow state for agents, suppliers, staffing, availability, marketing, visa cases, and approvals
- Internal audit logs, approval decisions, relationship notes, KPI notes, report definitions, and operational uploads
- Session-based browser authentication and the CT2 browser/API permission model

What CT2 does not own

- CT1 or other upstream booking-origin records
- Customer master data managed in external systems
- External payment platforms and authoritative finance ledgers
- Third-party identity or SSO ownership

> **Boundary rule:** when CT2 stores a booking ID, customer ID, supplier reference, or payment reference, it is storing a linkage pointer, not taking ownership of the source record.

3. Architecture And Runtime Model

Layer	Current implementation	Operational meaning
Present	Vanilla PHP views in `ct2_back_office/views/`	Browser-facing modules render directly through PHP without a framework

Layer	Current implementation	Operational meaning
ation		build step
Routing	`ct2_back_office/ct2_index.php`	Module and action are selected through query parameters such as `?module=auth&action=login`
Controllers	`ct2_back_office/controllers/`	Controllers coordinate request validation, permissions, model calls, and rendering
Models	`ct2_back_office/models/`	Models encapsulate MySQL persistence and reporting reads/writes
APIs	`ct2_back_office/api/`	JSON endpoints mirror protected CT2 operations for

Layer	Current implementation	Operational meaning
rface		scripted or AJAX use
Runtime bootstrap	`ct2_back_office/config/ct2_bootstrap.php`	Starts sessions, loads config, enforces shared helpers, CSRF, auth, and error handling
Database	MySQL or MariaDB seeded from `ct2_back_office/ct2_setup.sql`	Provides schema, roles, demo users, and baseline validation data
Writab	`storage/sessions/` and `storage/uploads/`	Must stay writable and outside version control; in cPanel they

Layer	Current implementation	Operational meaning
lestate		are linked from shared storage

4. Primary Modules And Their Operational Workflow

Module	Operational purpose	Typical process flow
Dashboard	Operational summary and queue visibility	Login, review counts and exceptions, identify pending work, drill into the owning module
Agents	Back-office registry of agency relationships and assignments	Create or update agent records, link staff assignments, review approvals, monitor bookings and

Module	Operational purpose	Typical process flow
		commissions
Suppliers	Partner onboarding, contracts, KPI tracking, and relationship notes	Review supplier readiness, update onboarding blockers, inspect contracts and KPIs, add notes, move records through approval
Availability	Packages, resource allocation, vehicle/driver dispatch, and seasonal or maintenance scheduling	Match resources to demand, dispatch assignments, record seasonal blocks, adjust availability constraints
Marketing	Campaign workflow, promotions, vouchers, affiliate tracking, and metrics	Prepare campaigns, review approval state, track clicks and redemptions, update notes, publish or revise offers
Fin	Snapshot reporting, reconciliati	Review report runs, inspect open flags,

Module	Operational purpose	Typical process flow
ancial	on flags, and export generation	update reconciliation state, export CSV for operational reporting
Visa	Visa application workflow, document checklist, payment tracking, and notifications	Intake cases, update checklist state, upload supporting files, monitor payments and appointment status, resolve exceptions
Staff	Internal staff records and assignment context	Maintain operating staff information, support agent and dispatch workflows, enforce role-aware visibility
Approvals	Decision queue for cross-module controlled changes	Review pending approvals, record decision notes, approve or reject, propagate final state to the owning module
A	Entry,	Authenticate

Module	Operational purpose	Typical process flow
auth	session lifecycle, and logout	through session-based login, rotate session identifiers, refresh CSRF state, terminate securely on logout

5. End-To-End Business Process Sequence

1. An operator signs in through the browser login route and lands on the dashboard.
2. The dashboard highlights pending operational items across agents, suppliers, dispatch, approvals, visa cases, and finance.
3. The operator enters the appropriate module, reviews seeded or live rows, and performs the required action.
4. State-changing operations are protected by session auth, CSRF validation, and permission checks.
5. Representative updates generate audit-log entries so operator actions remain traceable.
6. Where approval is required, the change is surfaced in the approvals queue and must be decided by an authorized role.
7. Uploads and exports use persistent runtime storage and must survive release switches.
8. Operational reporting and CSV output are treated as CT2 outputs, not as the master finance ledger.

6. Roles And Permission Model

CT2 uses a seeded role model to separate administrative, managerial, operational, and finance access. Browser access is session-based, while protected JSON endpoints require authentication and permission checks that mirror the same module boundaries.

Seeded user	Role	Primary use
ct2admin	system_admin	Full cross-module administration and validation
ct2manager	back_office_manager	Approvals, dashboard review, supplier and marketing management
ct2lead	team_lead	Supplier follow-up, availability planning, and operational coordination
ct2desk	front_desk_agent	Visa intake and front-office operational flows

Seeded user	Role	Primary use
ct2finance	accounting_staff	Financial review, reconciliation, and export operations

The current hardened contract requires browser and API authorization to stay aligned. Read actions must require the same view permissions as the equivalent browser module, and mutating actions must require the corresponding manage, decide, dispatch, or export permission as applicable.

7. Request, Session, API, And Audit Flow

Browser request lifecycle

1. The browser hits `ct2_index.php` with a target module and action.
2. Bootstrap loads configuration, current session state, CSRF helpers, and permission helpers.
3. The front controller maps the module to a controller class.
4. The controller validates access, reads or writes through the model layer, and renders the matching PHP view.
5. Unexpected browser exceptions are logged server-side and shown to the user as a generic error page, not raw exception text.

Authentication and session hardening

- Successful browser and API login rotates the session identifier.

- CSRF state is refreshed when auth state changes.
- Inactive users are denied consistently.
- Logout closes the current session context and starts a fresh unauthenticated session state.

API contract

- JSON endpoints live under `ct2_back_office/api/`.
- Responses must remain JSON-only, with no HTML or warning leakage.
- Endpoints inherit session-authenticated access and permission enforcement.
- `api.access` remains the baseline gate, with module-level permissions applied for read and write parity.

Audit and traceability

- Representative operational writes create `ct2_audit_logs` rows.
- Approval decisions, supplier updates, visa checklist changes, and financial flag updates are part of the hardened write surface.
- Operators should treat missing audit rows for successful writes as a regression.

8. Data Flow Between CT2 And External Systems

CT2 is designed to sit beside upstream systems rather than replace them. A booking may originate in CT1 or another source platform, while CT2 stores the back-office workflow that surrounds that booking: supplier readiness, resource allocation, dispatch context, approvals, visa progression, and financial reporting snapshots.

This means reconciliation work often crosses system boundaries. When an incident touches booking truth, customer truth, or external payments, the support team should capture the CT2 reference IDs and escalate to the system that owns the source data rather than trying to force CT2 into becoming the system of record.

9. Environment Setup And Branch Model

Local and validation environment

- PHP CLI with `pdo_mysql` is required.
- Local MySQL should be reachable through explicit TCP configuration.
- Node and Chrome or Chromium are required for browser-heavy validators.

- The seeded schema baseline comes from `ct2_back_office/ct2_setup.sql`.

Branch policy

- `develop` is the integration branch for ongoing CT2 work.
- `main` is the approved release branch.
- Deployments should be built from approved release state, not from unreviewed feature branches.

10. Quality Gate And CI

Every tracked repo change closes on the strict full-suite gate. CT2 does not allow formatter drift, lint findings, PHP warnings, HTML leakage in JSON, partial pass output, or skipped runtime/security validation without a documented blocker. Targeted checks can be used during debugging, but they do not replace the full suite.

Gate	Purpose
Format check	Prevents style drift
PHP lint	Blocks parse or syntax regressions
Structural smoke	Confirms expected file and app structure
DB smoke	Confirms runtime database connectivity and seed readiness
Browser accessibility	Checks keyboard focus and browser reachability
UI regression	Protects the shared shell, modal, toast, and split date/time contract
Load	Samples repeated timings for critical flows

Gate	Purpose
profile	
Route matrix	Proves breadth of browser and representative JSON routes
Runtime hardening	Checks write-path security, session, CSRF, and audit expectations
API POST regression	Validates representative mutating JSON endpoints
NFR sanity	Checks non-functional guardrails and representative timings
Role UAT	Confirms seeded role expectations across browser flows

Repository-level CI enforcement is handled by `.github/workflows/ct2_quality_gate.yml`. Release packaging and optional cPanel deployment are handled by `.github/workflows/ct2_cpanel_release_pipeline.yml`.

11. cPanel Release, Deployment, And Rollback

Release flow

1. GitHub Actions runs the strict suite on the release candidate.
2. A validated artifact is built with `ct2_release_artifact.sh`.
3. The artifact is uploaded to cPanel over SSH.
4. A new release directory is created under `<deploy-path>/releases/`.
5. Shared config and writable storage are linked into the new release.
6. Pre-live checks run on the target before the release is switched live.
7. The `current` symlink is advanced.
8. A live HTTP health check validates the public URL.
9. If the post-switch check fails, the workflow restores the previous release.

cPanel directory contract

Path type	Contract
Deploy root	`<deploy-path>/releases/<release-name>`
Active release	`<deploy-path>/current`
Shared config	`<deploy-path>/shared/config/ct2_local.php`
Shared uploads	`<deploy-path>/shared/storage/uploads/`
Shared sessions	`<deploy-path>/shared/storage/sessions/`
Shared pu	`CT2_CPANEL_PUBLIC_PATH` may point to `/home/<user>/public_html/ct2` for shared-domain deployment

Path type	Contract
blic mapping	

Current hosted model

CT2 is designed to coexist with multiple systems on a single cPanel account. The correct model is to keep CT2 releases isolated outside `public_html`, for example under `/home/<user>/apps/ct2`, and expose the active release through a public path such as `/home/<user>/public_html/ct2`. This keeps config, storage, and release lifecycle separate from other hosted applications.

Rollback scope

- Restore the previous `current` symlink or release pointer
- Restore the pre-deploy MySQL backup if schema or seeded data was changed
- Restore uploads if the failed release affected shared upload state
- Re-run post-deploy and live health checks before reopening access

12. Daily Operations And Support Workflow

1. Confirm login still works for an authorized account.
2. Confirm the dashboard loads without warnings or broken counts.
3. Check the approvals queue, visa workflow, and one representative JSON endpoint.
4. Verify exports still download and uploads still write to shared storage.
5. Review GitHub Actions and the latest release evidence before any change window.
6. Back up the database and uploads before every update deployment.

13. Troubleshooting Entry Points

Symptom	First checks
Database connection failure	Check `ct2_local.php`, verify host/port/user/password, confirm grants, run DB smoke
Login or session failure	Check writable sessions path, active-user status, and session rotation behavior
Upload failure	Check shared uploads path, file size limits, and visa document flow
API regression	Confirm JSON-only responses, auth, permission parity, and absence of warning leakage
Release failure	Inspect strict-suite logs, packaging step, SSH auth, shared config linkage, and live HTTP health check

14. Current Constraints And Open Operational Notes

- Windows XAMPP evidence remains a documented follow-up item even though the primary cPanel release path is active.
- Seeded accounts and demo data are for validation and UAT only and must not remain as production credentials.
- Any change that touches auth, API contracts, deployment scripts, or the shared UI shell should be assumed to require the full strict suite.
- cPanel deployment is artifact-based. Direct server-side branch pulls are outside the supported release contract.

15. Reference Documents

- `docs/ct2_integration_guide.md`

- docs/ct2_deployment_guide.md
- docs/ct2_operator_runbook.md
- docs/ct2_quality_gate.md
- docs/ct2_cpanel_release_flow.md
- docs/ct2_manual_qa_pack.md